ДЕПАРТАМЕНТ ОБРАЗОВАНИЯ И НАУКИ ГОРОДА МОСКВЫ
ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ПРОФЕССИОНАЛЬНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ГОРОДА МОСКВЫ
«Колледж связи № 54» имени П.М. Вострухина

ОТЧЕТ О ВЫПОЛНЕНИИ

по дисциплине

“Внедрение и поддержка компьютерных систем”

Практической работы №2

«Базовые CRUD-операции через консоль»

Выполнил: ст.гр. 3-ИСП11-43

ФИО: Юхимчук Евгений Викторович

Специальность: 09.02.07

Информационные системы и программирование

Проверил: преподаватель Кандыбаева

Москва

2026

Практическая работа №2. Базовые CRUD-операции через консоль

Ссылка на репозиторий: https://github.com/gangdexyuhi/client-server-practice
Папка работы: practice_02_client_server/

Цель работы

Отработать четыре базовые операции с данными — вставку (INSERT), чтение (SELECT), обновление (UPDATE) и удаление (DELETE) — в консольном приложении на Python и научиться защищаться от SQL-инъекций с помощью параметризованных запросов.

Используемые технологии

- Python 3.13, модуль sqlite3
- SQLite, файл app.db с таблицей tasks из работы №1
- SQL: INSERT, SELECT, UPDATE, DELETE с плейсхолдерами ?
- Свойства курсора lastrowid и rowcount

Структура проекта

```
src/
├── init_db.py        # создание таблицы (работа №1)
├── check_db.py       # проверка структуры
├── crud_console.py   # консольное CRUD-приложение
└── app.db            # база данных
```

Описание реализации

Приложение crud_console.py построено вокруг цикла меню в функции main(). Каждый пункт меню вызывает отдельную функцию, а соединение с базой открывает вспомогательная функция get_connection(), чтобы не дублировать код.

Таблица 1 — Функции приложения crud_console.py

Функция	Операция	SQL-запрос	Особенности
list_tasks()	R (Read)	SELECT id, title, status FROM tasks ORDER BY id	fetchall(); commit не нужен
add_task()	C (Create)	INSERT INTO tasks (title) VALUES (?)	Проверка на пустую строку, commit, lastrowid
mark_done()	U (Update)	UPDATE tasks SET status = 'done' WHERE id = ?	Проверка isdigit(), rowcount для «не найдено»
delete_task()	D (Delete)	DELETE FROM tasks WHERE id = ?	Подтверждение y/n, rowcount

Во всех запросах значения передаются вторым аргументом execute в виде кортежа, например (title,). Благодаря этому SQLite сам экранирует данные, и пользовательский ввод не может изменить структуру запроса.

```
def add_task():
    """CREATE: добавление новой задачи."""
    title = input("Название задачи: ").strip()
    if not title:
        print("Название не может быть пустым.")
        return

    conn = get_connection()
    cursor = conn.cursor()
    # параметризованный запрос: значение передаётся отдельно, в кортеже
    cursor.execute("INSERT INTO tasks (title) VALUES (?)", (title,))
    conn.commit()
    new_id = cursor.lastrowid   # id новой строки, берём до закрытия соединения
    conn.close()
    print(f"Задача добавлена, id = {new_id}")
```

Ход работы и результаты запуска

1. Запустил приложение, убедился, что список пуст, добавил задачи «Купить хлеб» и «Сделать уроки» и вывел их — обе в статусе new.

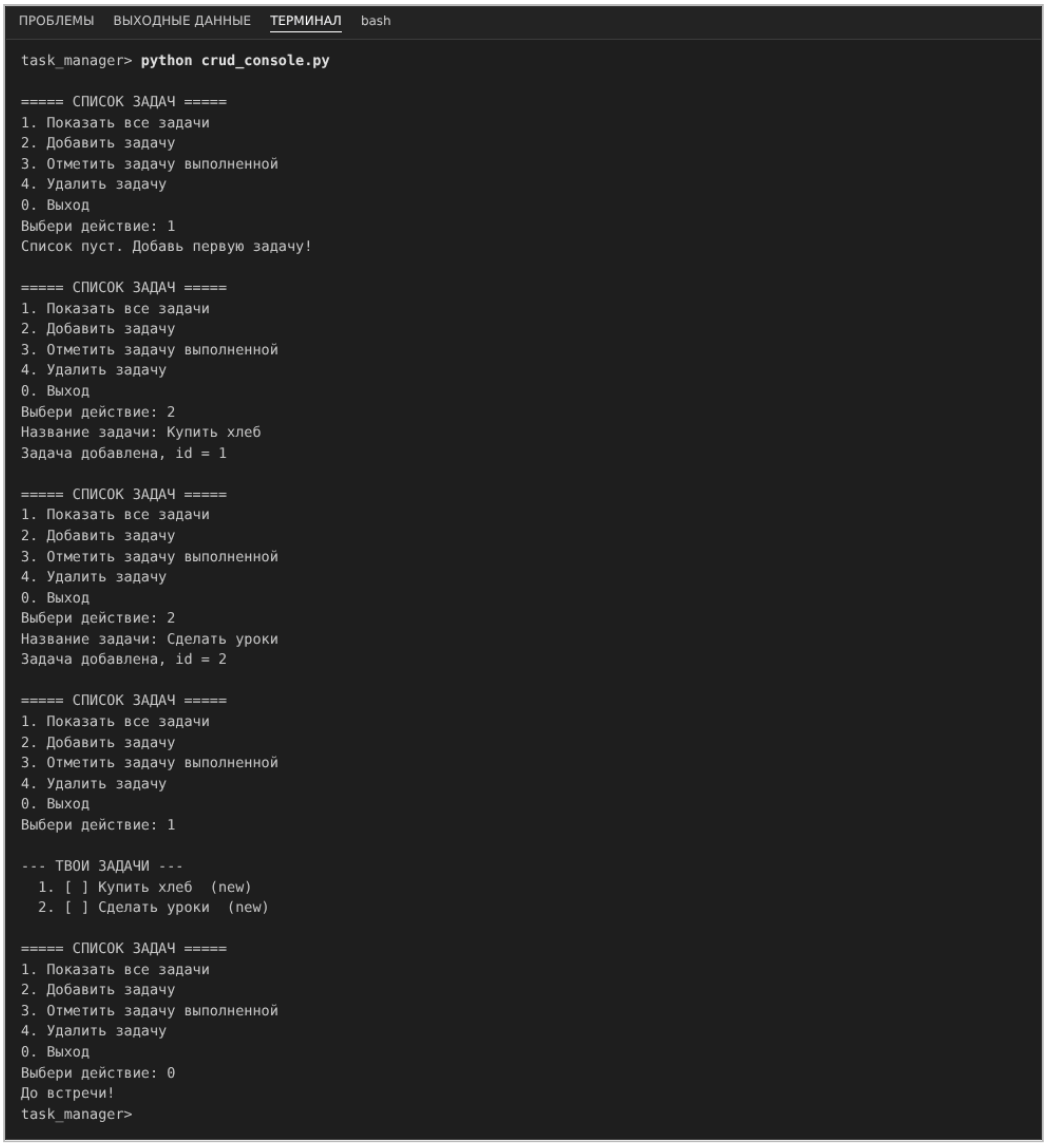


Рисунок 1 — Добавление задач и вывод списка

2. Отметил задачу № 1 выполненной и удалил задачу № 2 с подтверждением. В списке осталась одна задача со статусом done.

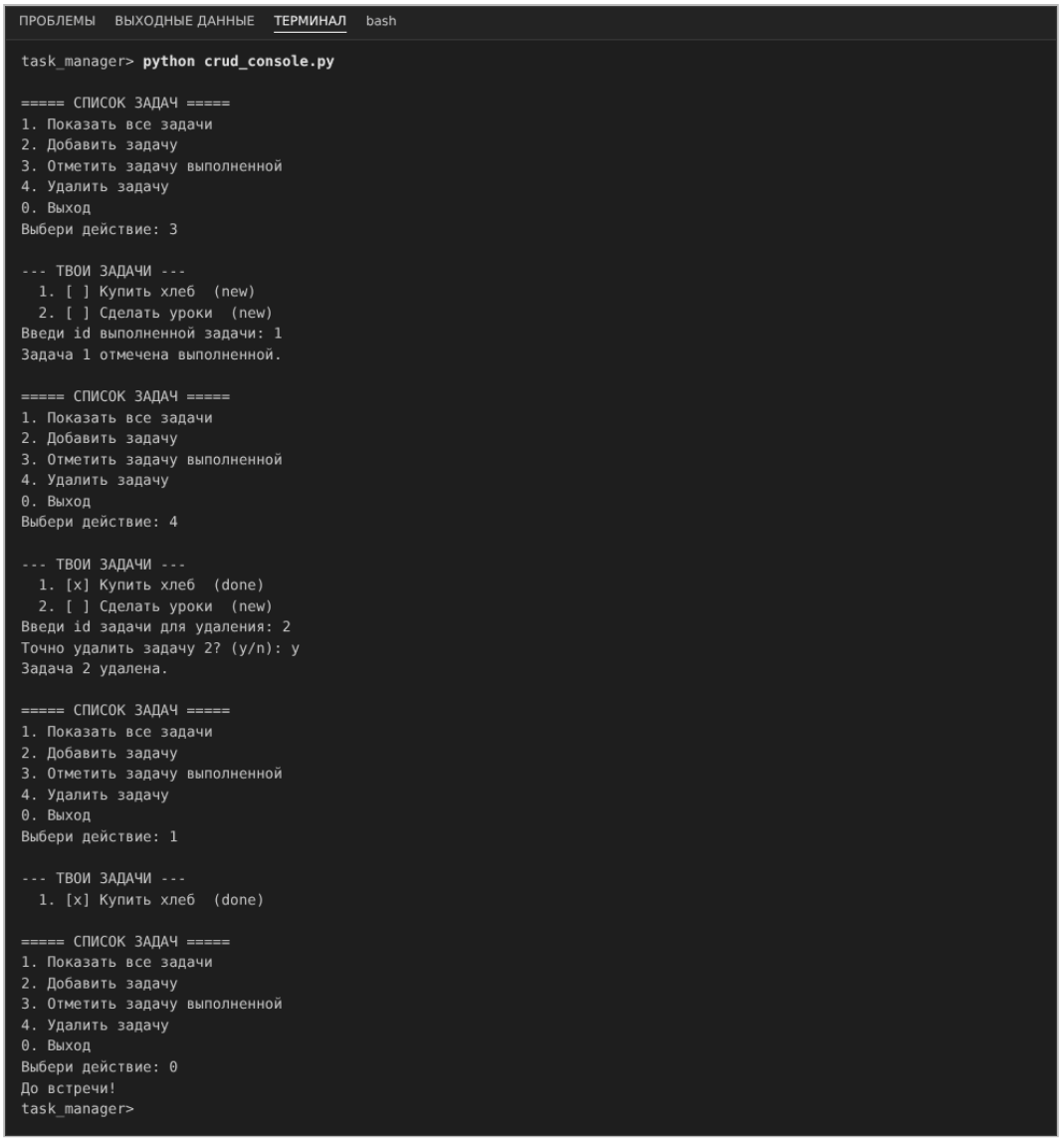


Рисунок 2 — Обновление статуса и удаление задачи

3. Перезапустил скрипт: данные на месте, значит commit() работает. Проверил обработку ошибок ввода: нечисловой id (abc), несуществующий id (99) и неверный пункт меню.

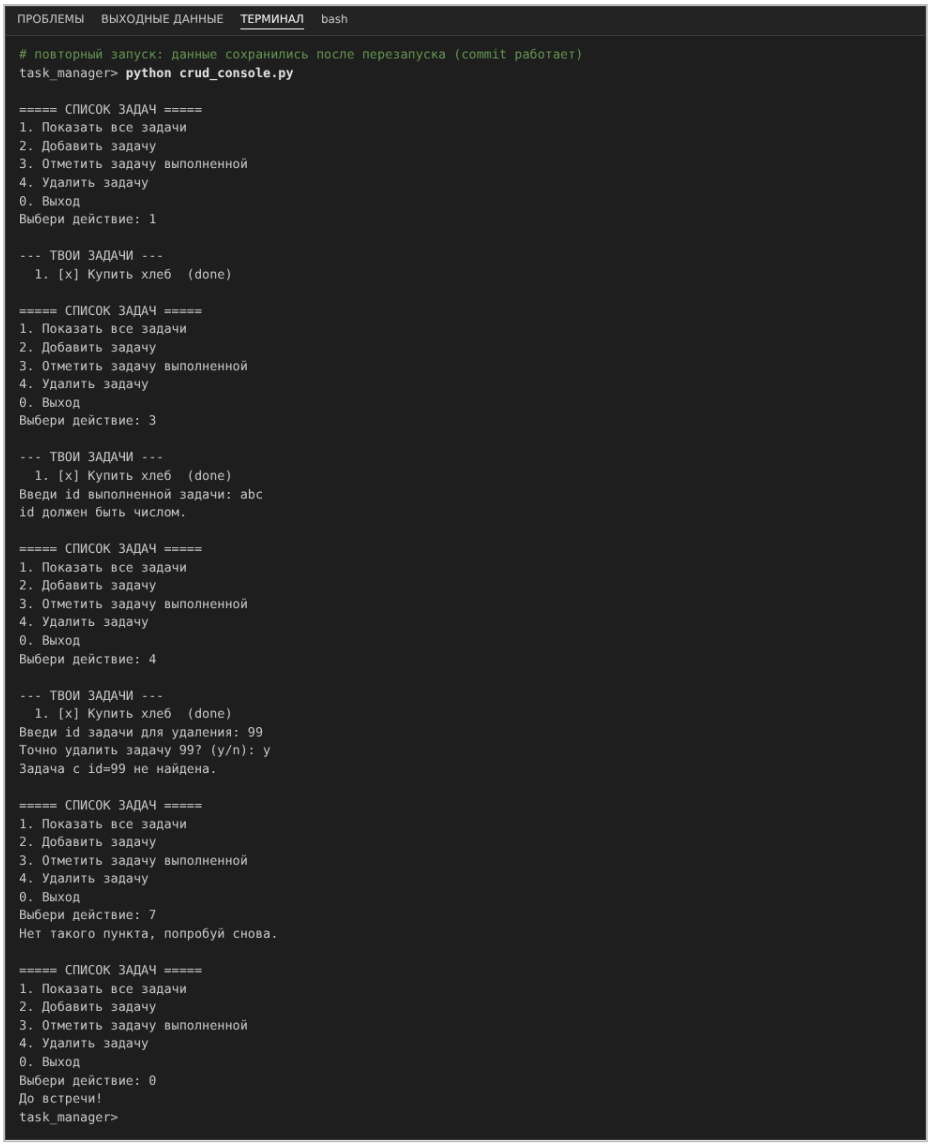


Рисунок 3 — Повторный запуск и обработка ошибок ввода

4. Эксперимент с SQL-инъекцией: ввёл название '); DROP TABLE tasks; -- . Строка сохранилась как обычный текст, таблица осталась целой — параметризованный запрос защищает базу.

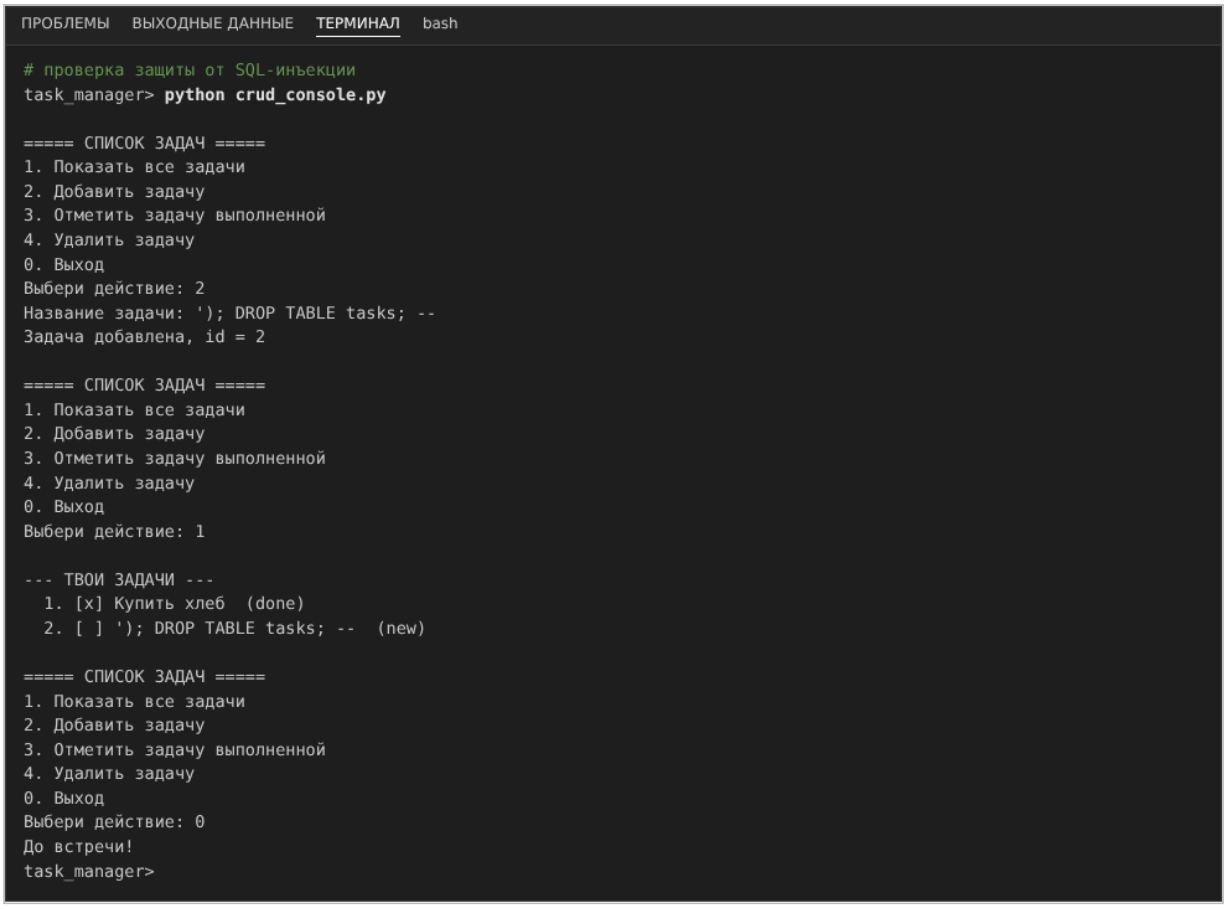


Рисунок 4 — Проверка защиты от SQL-инъекции

Примеры запросов

Таблица 2 — Запросы и ответы приложения

Действие пользователя	**Выполненный запрос**	**Ответ приложения**
Пункт 2, «Купить хлеб»	INSERT INTO tasks (title) VALUES (?) — ('Купить хлеб',)	Задача добавлена, id = 1
Пункт 3, id = 1	UPDATE tasks SET status = 'done' WHERE id = ? — (1,)	rowcount = 1 → Задача 1 отмечена выполненной
Пункт 4, id = 99	DELETE FROM tasks WHERE id = ? — (99,)	rowcount = 0 → Задача с id=99 не найдена
Пункт 3, ввод abc	запрос не выполняется	id должен быть числом

Ответы на контрольные вопросы

1. Почему для SELECT не нужен commit(), а для INSERT — нужен?

SELECT только читает данные и ничего не меняет в базе, поэтому фиксировать нечего. INSERT изменяет данные: изменение сначала

происходит внутри транзакции и записывается в файл только после commit(). Без него новая строка пропадёт при закрытии соединения.

2. Что вернёт fetchone(), если в таблице нет строк?

None.

3. Чем fetchone() отличается от fetchall() и когда какой использовать?

fetchone() возвращает одну следующую строку результата (кортеж) или None, fetchall() — список всех оставшихся строк. fetchone() удобен, когда ожидается одна запись, например поиск по id, а fetchall() — когда нужно вывести весь список.

4. Почему (title,) — это кортеж, а (title) — нет?

Кортеж в Python определяется запятой, а не скобками. (title) — это просто значение title в скобках, а (title,) — кортеж из одного элемента. execute ожидает последовательность параметров, поэтому без запятой строка была бы воспринята как набор отдельных символов, и возникла бы ошибка Incorrect number of bindings.

5. Что вернёт cursor.rowcount после UPDATE, если такого id нет?

0 — запрос выполнится без ошибки, но не изменит ни одной строки. По этому значению программа понимает, что задача не найдена.

6. Зачем проверять isdigit() перед int(input())?

Если пользователь введёт не число (например, abc), int() выбросит ValueError, и программа аварийно завершится. Проверка isdigit() позволяет заранее отсечь некорректный ввод и вывести понятное сообщение.

7. Как параметризованные запросы защищают от SQL-инъекции?

Значения передаются в базу отдельно от текста SQL-запроса. SQLite подставляет их как данные, а не как часть команды, поэтому кавычки и символы вроде ; или -- внутри значения не меняют смысл запроса. Строка "'); DROP TABLE tasks; --" сохранилась как обычное название задачи.

Вывод

В ходе работы я реализовал консольное приложение с полным набором CRUD-операций над таблицей tasks. Я закрепил порядок работы с транзакцией (подключение → запрос → commit → закрытие), научился использовать lastrowid и rowcount, обрабатывать некорректный ввод и

защищать базу от SQL-инъекций параметризованными запросами. Перезапуск программы подтвердил, что данные сохраняются в файле.